UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 8 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Computer generations and history

Source: CSC429 (7).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The generation described is the Fourth Generation of computers (approximately 1971–present), characterised by microprocessors, GUI/mouse technology, and the growth of computer networks leading to the internet.

(b) The TWO main technologies available in this generation are: (1) Microprocessor / VLSI (Very Large Scale Integration) integrated circuits — the CPU on a single chip, enabling personal computers; and (2) Graphical User Interface (GUI) with mouse/pointing-device technology (supported by networking technology).

(c) TWO main features of this generation: (1) Use of microprocessors/VLSI chips giving small, cheap, powerful personal computers; (2) GUI and mouse-based interaction making computers user-friendly; also (3) development of computer networks and the internet for resource/data sharing. Any two of these are acceptable.

Unofficial marking guide:

- *1 mark: (a) correctly names the Fourth Generation.*
- *2 marks: (b) one mark each for two correct technologies (microprocessor/VLSI; GUI/mouse; networking).*
- *2 marks: (c) one mark each for two correct features (microprocessor-based PCs; GUI/mouse user-friendliness; networking/internet).*

Model confidence: high.

Model note: Question text has minor OCR/grammar issues ('remark' for 'mark'); interpreted conservatively. Generation numbering follows the standard CSC429 five-generation classification.

QUESTION A2 (5 marks)

Topic: Data representation and number systems

Answer: left 86 -> 0 1010110; right -30 -> 1 0011110; 86 - (-30) = 116 -> sign-magnitude 0 1110100

- *1 mark for converting both operands to sign-magnitude.*
- *1 mark for applying the sign rule for subtraction.*
- *1 mark for the correct magnitude operation.*
- *1 mark for the correct sign bit.*
- *1 mark for the final decimal and byte representation.*

QUESTION A3 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The technique is direct addressing (also called absolute addressing). The instruction's address field A contains the actual/effective address of the operand, so the operand is fetched directly from memory location A — the figure shows the address field pointing straight into memory to the operand, with no intermediate pointer.

(b) How it is used: the instruction is fetched and decoded; the CPU takes the address field A of the instruction and places it on the address bus as the effective address (EA = A); memory is read at that location and the operand is transferred to the CPU (e.g. into a register or the ALU). No further address calculation or extra memory access for an address is needed — one operand fetch from memory at address A. Example: LOAD 500 fetches the contents of memory location 500 into the accumulator.

(c) Advantage: simple and fast — only one memory reference is needed to get the operand (no extra memory access to compute an address), and no address-calculation hardware is required. Disadvantage: limited addressable range (the address field must be large enough to hold a full memory address, making instructions long) and it is inflexible — the address is fixed at assembly time, so it cannot easily support relocation, arrays/pointers or dynamic data structures.

© Hak Cipta Universiti Teknologi MARA

Unofficial marking guide:

• 1 mark: correctly names the technique as direct addressing (absolute addressing).

• 1 mark: states that the address field A is the effective address of the operand (EA = A).

• 1 mark: explains the fetch sequence — instruction decoded, address A sent to memory, operand read directly from location A into the CPU.

• 1 mark: gives one valid advantage (e.g. simple/fast, only one memory access, no address calculation).

• 1 mark: gives one valid disadvantage (e.g. limited address range/long instruction format, inflexible/fixed address, poor support for relocation or data structures).

Model confidence: high.

Model note: Figure 2 is only available as extracted text; the answer assumes the diagram shows the instruction's address field A pointing directly to the operand in memory (the standard direct-addressing depiction). If the figure instead shows A pointing to a memory word that itself holds the operand's address, the technique would be indirect addressing — this must be confirmed from the figure.

QUESTION A4 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (5).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

The question asks for completion of a taxonomy of parallel processor architecture with appropriate examples. The standard taxonomy (Flynn-style / SISD–SIMD–MISD–MIMD, often extended with multiprocessor vs multicomputer subdivisions) is completed as follows, with one example per class:

SISD (Single Instruction, Single Data): a conventional uniprocessor — e.g. a single-core Pentium/ARM Cortex-M CPU executing one instruction stream on one data stream.

SIMD (Single Instruction, Multiple Data): one control unit drives many processing elements on different data — e.g. array/vector processors such as the Cray-1 vector unit, MMX/SSE/AVX SIMD extensions in x86, or a GPU's warps executing the same instruction over many lanes.

MISD (Multiple Instruction, Single Data): several processing units apply different instructions to the same data stream — e.g. a systolic array or fault-tolerant pipeline with replicated processors comparing the same input (rarely used in practice).

MIMD (Multiple Instruction, Multiple Data): independent processors execute different instruction streams on different data — e.g. a multicore CPU (Intel Core i7), a shared-memory multiprocessor (SMP), or a cluster/network of workstations.

Within MIMD the taxonomy is usually further divided into: shared-memory multiprocessors (tightly coupled, e.g. SMP with a common bus) and message-passing multicomputers (loosely coupled, e.g. a Beowulf cluster or Intel Paragon); and by memory access into UMA (uniform memory access, e.g. SMP) versus NUMA (non-uniform memory access, e.g. SGI Origin).

Note: the exact branch labels to be filled in are those shown in Figure CSC429-2016-06-D1; the classes and examples above correspond to the standard Flynn taxonomy that the figure depicts.

Unofficial marking guide:

• 1 mark: correctly identifies SISD with a valid uniprocessor example

• 1 mark: correctly identifies SIMD with a valid example (vector/array processor or SIMD extensions/GPU)

• 1 mark: correctly identifies MISD with a valid example (systolic array / replicated pipeline)

• 1 mark: correctly identifies MIMD with a valid example (multicore/SMP/cluster)

• 1 mark: correct subdivision of MIMD into shared-memory multiprocessor vs message-passing multicomputer (or UMA/NUMA) with example

Model confidence: medium.

Model note: The figure CSC429-2016-06-D1 is only available as an asset reference; the exact blank labels in the taxonomy diagram could not be read, so the answer assumes the standard Flynn SISD/SIMD/MISD/MIMD taxonomy with MIMD subdivisions. If the figure uses different branch names, map the examples to those labels.

QUESTION A5 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (3).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two types of characters provided by the keyboard [4 marks]

1. Printable (graphic/visible) characters. These are characters that have a visible glyph and are normally

echoed to the display, e.g. letters (A–Z, a–z), digits (0–9), punctuation marks and symbols such as @, #, %, and the space character. When such a key is pressed the keyboard's encoder produces the corresponding character code (e.g. its ASCII code) which is sent to the I/O module and then to the processor/memory for processing or display.

2. Control (non-printable) characters. These do not produce a visible symbol but cause an action or control function, e.g. Enter/Carriage Return (CR), Line Feed (LF), Tab, Backspace, Escape, Delete, and cursor-control keys. They are represented by control codes (e.g. ASCII codes 0–31) and are interpreted by the software/driver to perform an operation such as starting a new line or moving the cursor.

(b) Three categories of input/output module, with an example each [6 marks]

1. Human-readable / human-machine interface modules. These convert between the machine's electrical signals and a form a person can perceive or produce, e.g. a keyboard or display (monitor) — the keyboard is a human-readable input module and the monitor a human-readable output module.

2. Machine-readable modules. These interface the computer to equipment that reads/writes machine-oriented media without human interpretation, e.g. a magnetic disk drive or a barcode/optical scanner reading data directly into the system.

3. Communication (data-transfer) modules. These allow the computer to exchange data with a remote device or another computer over a link, e.g. a network interface card (NIC)/modem connecting the computer to a network or telephone line.

(Each category should be named, briefly described in terms of its role in interfacing the processor/memory to the external device, and illustrated with a valid example.)

Unofficial marking guide:

- *1 mark: correctly naming/identifying the first type of keyboard character (printable/graphic).*
- *1 mark: explaining the first type with a valid example (letters, digits, symbols echoed to display).*
- *1 mark: correctly naming/identifying the second type (control/non-printable).*
- *1 mark: explaining the second type with a valid example (Enter, Tab, Backspace, cursor keys).*
- *1 mark: naming and describing category 1 (human-readable) of I/O module.*
- *1 mark: giving an appropriate example for category 1 (keyboard, monitor).*
- *1 mark: naming and describing category 2 (machine-readable) of I/O module.*
- *1 mark: giving an appropriate example for category 2 (disk drive, scanner).*
- *1 mark: naming and describing category 3 (communication) of I/O module.*
- *1 mark: giving an appropriate example for category 3 (NIC, modem).*

Model confidence: high.

Model note: The question text is slightly ambiguous ('types of characters'); the answer assumes the standard printable vs control-character distinction. Category names follow the common textbook classification (human-readable, machine-readable, communication); other valid classifications (e.g. by data direction) would be accepted if consistently described with examples.

QUESTION A6 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (14).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) A maskable interrupt is a hardware interrupt that the CPU can temporarily ignore or postpone by setting/clearing an interrupt mask bit in its interrupt-enable register (or by the interrupt's priority mask). While masked, the interrupt request is held pending and is not serviced until it is unmasked; it is not lost. Maskable interrupts allow the programmer to protect critical sections and to control which interrupts may preempt the current handler. They are distinguished from non-maskable interrupts (NMI), which cannot be disabled and are used for catastrophic events.

(b) Defined (fixed) priority: a higher-priority interrupt may preempt a lower-priority handler, but a lower-priority request must wait. Reading Figure 2 as the main program being interrupted first by Handler A, then B, then C, then D, then E, with priority A > B > C > D > E, the execution sequence is: Main program → A starts → B interrupts A (B runs to completion) → A resumes and completes → C starts → D interrupts C (D runs to completion) → C resumes → E interrupts C (E runs to completion) → C resumes and completes → return to Main program. That is: Main → A → B → A → C → D → C → E → C → Main. (The exact nesting order must be confirmed from the arrows in Figure 2.)

(c) Disable-interrupt method: once a handler begins, interrupts are disabled, so no other handler can preempt it; each handler runs to completion before the next pending request is served. The sequence is therefore strictly serial: Main → A (complete) → B (complete) → C (complete) → D (complete) → E

© Hak Cipta Universiti Teknologi MARA

(complete) → Main. No nesting occurs.

(d) Interrupt C finishes earlier under the disable-interrupt method. Justification: with interrupts disabled, C runs non-stop from start to finish, so its completion time equals its own service time. Under defined priority, C is preempted twice (by D and by E), so C's completion is delayed by the full service times of D and E plus the context-switch overhead. Hence C finishes earlier when interrupts are disabled (at the cost of delaying D and E).

Unofficial marking guide:

- *1 mark: states a maskable interrupt can be disabled/ignored by the CPU via an interrupt mask/enable bit*
- *1 mark: explains it is held pending and serviced when unmasked (contrast with non-maskable interrupt)*
- *1 mark: correct starting point (main program) and first handler A*
- *1 mark: correct nesting of B within A (B preempts A, A resumes)*
- *1 mark: correct nesting of D and E within C according to priority*
- *1 mark: correct return to main program / overall ordering consistent with Figure 2*
- *1 mark: states interrupts are disabled during a handler so no preemption occurs*
- *1 mark: correct serial sequence Main → A → B → C → D → E → Main*
- *1 mark: identifies disable-interrupt method as finishing C earlier*
- *1 mark: justifies that C is not preempted (runs continuously) whereas under priority C is delayed by D and E*

Model confidence: medium.

Model note: Figure 2 is only available as extracted text; the assumed nesting order (A first, then B, C, D, E with priority A>B>C>D>E) is inferred from the labels and must be verified against the arrows in the actual figure. If the figure shows a different arrival order or priority, the sequences in (b) and (c) should be re-ordered accordingly; the reasoning in (d) is unaffected.

QUESTION A7 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (13).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Registers are small, high-speed storage locations inside the CPU used to hold data, instructions and addresses temporarily during program execution. Their specific functions are: (1) Program Counter (PC): holds the address of the next instruction to be fetched; it is automatically incremented after each fetch so that instructions are executed in sequence unless a branch changes it. (2) Instruction Register (IR): holds the instruction currently being decoded and executed; its contents are fed to the control unit to generate control signals. (3) Memory Address Register (MAR): holds the address of the memory location (or I/O port) that is to be read from or written to; it connects the CPU to the address bus. (4) Memory Data Register (MDR) / Memory Buffer Register (MBR): holds the data word just read from memory or the data word to be written to memory; it acts as a buffer between the CPU and the data bus. (5) Accumulator (AC): a general-purpose working register that holds one operand and the result of arithmetic/logic operations performed by the ALU. (6) General-Purpose Registers (GPRs): a set of registers used by the programmer to hold operands, intermediate results, pointers and index values; they reduce the number of slow memory accesses. (7) Status/Flag Register (Condition Code Register): holds individual bits (flags) such as zero, carry, sign, overflow and interrupt-enable that record the outcome of the last ALU operation and control conditional branching. (8) Stack Pointer (SP): holds the address of the top of the stack in memory, used for subroutine calls, returns and interrupt handling. (9) Index Register: holds an offset that is added to a base address to form an effective address for indexed addressing. (10) Instruction/Data temporary registers (e.g. TMP, A, B) inside the CPU: hold operands between pipeline stages or during multi-step operations. In summary, address registers (PC, MAR, SP, index) locate information, data registers (MDR, AC, GPRs) hold information, the IR holds the instruction, and the status register records processor state.

Unofficial marking guide:

- *1 mark: correct definition of a register as fast CPU-internal storage*
- *1 mark: Program Counter (PC) – address of next instruction*
- *1 mark: Instruction Register (IR) – current instruction for decoding*
- *1 mark: Memory Address Register (MAR) – address on the address bus*
- *1 mark: Memory Data Register / MBR – data buffer on the data bus*
- *1 mark: Accumulator – ALU operand/result*
- *1 mark: General-purpose registers – programmer-visible operands/results*
- *1 mark: Status/Flag register – condition codes and control flags*
- *1 mark: Stack Pointer – top-of-stack address for calls/interrupts*

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Index register or temporary registers – effective address/operand holding

Model confidence: high.

Model note: The question text contains an OCR spacing error ('specificfunction'); corrected to 'specific function'. No figure or table is required. The list is representative; any eight to ten correctly described register types with distinct functions should earn full credit.

QUESTION A8 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (10).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Four storage access methods with examples: (1) Sequential access — data is read in a fixed linear order; example: magnetic tape. (2) Direct access — data is accessed by jumping to a block/record and then searching sequentially within it; example: hard disk drive (or floppy disk). (3) Random access — any location can be accessed directly in roughly the same time; example: main memory (RAM) or ROM. (4) Associative access — data is searched by content rather than by address; example: cache memory (associative cache) or content-addressable memory (CAM). [6 marks]

(b) With 11 address bits, the number of addressable memory locations is 2^11 = 2048 locations. [2 marks]

(c) 2 MB = 2 × 2^20 bytes = 2^21 bytes. If each addressable unit is one byte, the number of addresses is 2^21, so the number of address bits required is log2(2^21) = 21 bits. [2 marks]

Unofficial marking guide:

• 1 mark: correct identification of sequential access method
• 1 mark: correct example for sequential access (e.g., magnetic tape)
• 1 mark: correct identification of direct access method
• 1 mark: correct example for direct access (e.g., hard disk)
• 1 mark: correct identification of random access method
• 1 mark: correct example for random access (e.g., RAM)
• 1 mark: correct identification of associative access method
• 1 mark: correct example for associative access (e.g., cache/CAM)
• 1 mark: correct calculation 2^11 = 2048 locations
• 1 mark: correct final answer 2048
• 1 mark: correct conversion 2 MB = 2^21 bytes
• 1 mark: correct calculation of address bits = 21 bits

Model confidence: high.

Model note: Assumption for part (c): memory is byte-addressable, so each byte has a unique address. If word-addressable with a different word size, the answer would change. Part (a) examples are typical; other valid examples may be accepted.

QUESTION A9 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (16).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The two machine-cycle processes governed by main memory are the Fetch and Memory (data transfer) / Indirect processes — most standard treatments (e.g. Stallings) identify the memory-governed phases as the fetch cycle and the memory read/write (data transfer) operations within the execute/indirect phases. In the Fetch process, the CPU places the address held in the PC on the address bus and issues a memory-read control signal; main memory decodes the address, accesses the addressed word, and returns the instruction on the data bus into the IR (then the PC is incremented). In the Memory/Data-transfer process (memory read or write during Execute, or the Indirect cycle), the CPU supplies an effective address (often from the MAR) and a read or write command; main memory either delivers the operand to the MDR/data bus (read) or stores the value presented on the data bus (write). In both cases main memory governs the timing: the CPU must wait for the memory access to complete (memory access time / wait states) before it can use the data, so these are the phases whose duration and sequencing are dictated by main-memory operation rather than by internal ALU/register activity.

(b) Two operations comprised in the Execute phase (any two): data transfer between CPU registers and memory or I/O (load/store); arithmetic/logic operations performed by the ALU (add, subtract, AND, OR, etc.); control transfer (branch/jump and conditional branch, altering the PC); I/O operations; and register-to-register transfers / incrementing the PC. (Any two listed.)

© Hak Cipta Universiti Teknologi MARA

(c) During the Fetch cycle: the Program Counter (PC) holds the address of the next instruction to be fetched. Its contents are transferred to the Memory Address Register (MAR) and placed on the address bus so main memory can be addressed. Once the instruction word has been read from memory into the Memory Data Register (MDR) and then into the Instruction Register (IR), the PC is incremented (PC ← PC + 1, or + instruction length) so that it points to the next instruction/operand in sequence. The Instruction Register (IR) receives and holds the fetched instruction word; its opcode field is passed to the control unit/instruction decoder to determine which operation is to be performed, and its address/operand fields are used later in the execute (or indirect) phase. Thus the PC supplies the address and is advanced, while the IR latches the fetched instruction for decoding.

Unofficial marking guide:

• 1 mark: naming the Fetch process as one memory-governed process
• 1 mark: explaining that in Fetch the PC/MAR address is sent to memory and the instruction is read back into the IR/MDR
• 1 mark: naming the Memory (data transfer/read-write) or Indirect process as the second memory-governed process
• 1 mark: explaining that in this process the CPU issues a read or write with an effective address and memory supplies/stores the operand, with timing set by memory access time
• 1 mark: first valid Execute-phase operation listed (e.g. data transfer, ALU arithmetic/logic, control transfer, I/O)
• 1 mark: second valid Execute-phase operation listed
• 1 mark: PC holds address of next instruction and its contents are passed to MAR/address bus
• 1 mark: PC is incremented after the fetch so it points to the next instruction
• 1 mark: IR receives/holds the fetched instruction word
• 1 mark: IR opcode is passed to control unit/decoder to determine the operation (and operand fields used later)

Model confidence: high.

Model note: Part (a) wording is slightly ambiguous; the answer assumes the standard Stallings-style machine cycle in which Fetch and the memory data-transfer (read/write, including indirect) processes are the memory-governed phases. If the lecturer intends 'Fetch and Indirect' specifically, the same explanation applies with the Indirect cycle emphasised. No figure was required.

QUESTION A10 (10 marks)

Topic: Data representation and number systems

a) Answer: 256610 = A0616

• 1 mark for repeated division by 16.
• 1 mark for correct remainders.
• 1 mark for reading remainders in reverse order.
• 1 mark for the correct hexadecimal digits.
• 1 mark for the base subscript.

b) Answer: 7E816 = 202410

• 1 mark for place values 16^2, 16^1 and 16^0.
• 1 mark for correct digit values.
• 1 mark for correct products.
• 1 mark for correct addition.
• 1 mark for the final decimal value.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (18).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption: Figure 2 is interpreted as y = A - ((2C * D) / (B + A^2)). The expression is evaluated as: temp1 = 2*C; temp2 = temp1*D; temp3 = A*A; temp4 = B + temp3; temp5 = temp2 / temp4; y = A - temp5. Registers A, B, C, D are fixed; x, y, z are temporary. Final result must be in y.

One-address program (accumulator-based):

```
LOAD A       ; ACC = A
MUL A        ; ACC = A^2
ADD B        ; ACC = A^2 + B
STOR z       ; z = A^2 + B
LOAD C       ; ACC = C
MUL D        ; ACC = C*D
ADD C        ; ACC = 2*C*D  (since C*D + C*D = 2*C*D)
DIV z        ; ACC = (2*C*D)/(A^2+B)
STOR x       ; x = (2*C*D)/(A^2+B)
LOAD A       ; ACC = A
SUB x        ; ACC = A - x
STOR y       ; y = final answer
```

Two-address program (operand1 = operand1 op operand2):

```
MOV x, C     ; x = C
ADD x, C     ; x = 2C
MUL x, D     ; x = 2C*D
MOV z, A     ; z = A
MUL z, A     ; z = A^2
ADD z, B     ; z = B + A^2
DIV x, z     ; x = (2C*D)/(B + A^2)
MOV y, A     ; y = A
SUB y, x     ; y = A - x
```

Three-address program (dest = src1 op src2):

```
MUL x, C, D   ; x = C*D
ADD x, x, x   ; x = 2*C*D
MUL z, A, A   ; z = A^2
ADD z, B, z   ; z = B + A^2
DIV x, x, z   ; x = (2*C*D)/(B + A^2)
SUB y, A, x   ; y = A - x
```

Unofficial marking guide:

- *1 mark: correct interpretation of Figure 2 as y = A - ((2C*D)/(B + A^2)) and correct order of operations.*
- *6 marks: one-address program – 1 mark for correct use of LOAD/STOR, 1 mark for computing A^2+B, 1 mark for computing 2C*D, 1 mark for division, 1 mark for final subtraction, 1 mark for storing final answer in y.*
- *6 marks: two-address program – 1 mark for MOV to initialise, 1 mark for computing 2C, 1 mark for 2C*D, 1 mark for A^2+B, 1 mark for division, 1 mark for final subtraction into y.*
- *6 marks: three-address program – 1 mark for MUL x,C,D, 1 mark for doubling to 2C*D, 1 mark for MUL z,A,A, 1 mark for ADD z,B,z, 1 mark for DIV x,x,z, 1 mark for SUB y,A,x.*
- *1 mark: correct assembly notation (mnemonics and operand order) consistent with Table 1 for all three programs.*

Model confidence: medium.

*Model note: Figure 2 is not fully legible from the extracted text; the expression is assumed to be y = A - (2C*D)/(B + A^2). If the figure shows a different grouping (e.g., denominator B + A^2 or (B+A)^2), the programs must be adjusted accordingly. Table 1 uses inconsistent punctuation (e.g., ADD (p.q.r)); operand order is taken as dest, src1, src2 for three-address and dest, src for two-address. One-address instructions are assumed to use an implicit accumulator.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA